Risk Alert Classifier

Theory Concepts & Definitions | Part A of the project (Supervised Learning)

1. Logistic Regression

Definition. Logistic Regression is a supervised *linear* model for classification. It computes a weighted sum of the features, $z = b_0 + b_1x_1 + ... + b_nx_n$, and squeezes it through the sigmoid function $\text{sigmoid}(z) = 1 / (1 + e^{-z})$ to obtain a probability $P(y=1 \mid x)$ between 0 and 1. The coefficients are learned by minimising the **log-loss** (cross-entropy). A threshold (default 0.5) converts the probability into a class label.

Why it is suitable for classification.

- It outputs **probabilities**, so the alert threshold can be tuned to the business (important for risk scoring).
- Fast to train, stable on small and medium data and a very strong baseline.
- Coefficients are interpretable (change in log-odds per unit of a feature) - important for regulated banking decisions.
- Works very well when the classes are (almost) linearly separable, as in this dataset.

2. Classification performance metrics - why accuracy alone is insufficient

All basic metrics come from the **confusion matrix**:

	Predicted Low Risk (0)	Predicted High Risk (1)
Actual Low Risk (0)	True Negative (TN)	False Positive (FP) = Type-I error
Actual High Risk (1)	False Negative (FN) = Type-II error	True Positive (TP)

Accuracy = (TP + TN) / (TP + TN + FP + FN) is the share of correct predictions. With about 12 % high-risk customers, a model that *always* predicts 'Low Risk' is about 88 % accurate yet detects **no** risky customer. This is the **accuracy paradox**. Therefore accuracy must be complemented by precision, recall, F1, FPR, the confusion matrix and ROC-AUC, which describe how well the *minority* class is handled.

3. Type-I and Type-II errors in risk prediction

Error	Statistical name	Meaning for the bank	Typical cost
Type-I	False Positive (FP)	A safe customer is flagged as high-risk.	Wasted manual review, possible annoyance of a good customer.
Type-II	False Negative (FN)	A truly high-risk customer is missed.	Undetected default or fraud - direct financial loss. Usually the more expensive error.

Because Type-II errors cost more, this project selects the final model by **minimising false negatives**.

4. Precision, Recall, F1-Score, TPR and FPR

Metric	Formula	Question it answers
Precision	TP / (TP + FP)	Of all customers flagged high-risk, how many really are?

Metric	Formula	Question it answers
Recall = TPR (sensitivity)	TP / (TP + FN)	Of all truly high-risk customers, how many did we catch?
F1-Score	2 x Precision x Recall / (Precision + Recall)	Harmonic mean - one number balancing precision and recall.
F2-Score	5 x P x R / (4 x P + R)	Like F1 but recall counts twice as much (used for tuning here).
FPR	FP / (FP + TN)	Of all safe customers, how many were wrongly flagged?
Specificity (TNR)	TN / (TN + FP) = 1 - FPR	Of all safe customers, how many were correctly cleared?

5. AUC-ROC

The **ROC curve** plots the True Positive Rate (y-axis) against the False Positive Rate (x-axis) for *every possible decision threshold*. The **AUC** (area under the curve) summarises it in one number between 0 and 1: 0.5 means random guessing, 1.0 means perfect ranking. It can be read as the probability that a randomly chosen high-risk customer receives a higher score than a randomly chosen low-risk customer.

How it helps: it evaluates a classifier independently of the threshold, allows fair comparison of different algorithms, and is less sensitive to class imbalance than accuracy. With extreme imbalance it is best combined with a Precision-Recall curve.

6. Why imbalanced data is a problem

In imbalanced data one class (here 'High Risk', about 12 %) is rare. Learning algorithms minimise the overall error, so they can achieve a low error by mostly predicting the majority class. Consequences:

- The minority class provides few examples, so the decision boundary is poorly learned and drifts towards the minority class.
- **Recall** of the minority class is low although **accuracy** looks high (accuracy paradox).
- Standard train/test splits can distort the minority share - use **stratified** splitting.

Remedies: resampling (under-sampling, over-sampling, SMOTE, ADASYN), class weights, threshold moving, and suitable metrics (recall, F1, AUC, PR-curve).

Glossary of other concepts used in the project

Concept	Definition
Stratified train-test split	Splitting that keeps the class proportions identical in the training and test sets (`stratify=y`).
Data leakage	Information from the test data influencing training (e.g. fitting an imputer on all data). Avoided by fitting preprocessing on the training split only.
KNN Imputer	Multivariate imputation: a missing value is replaced by the average of that feature in the k most similar rows (distance measured on the other, scaled features).
Standardisation	Rescaling a feature to mean 0 and standard deviation 1; necessary for distance-based methods such as KNN.
One-hot encoding	Converting a categorical column into binary 0/1 columns, one per category.
Random Under-Sampling	Randomly removes majority-class samples until the classes are balanced. Fast, but discards information.
Random Over-Sampling	Randomly duplicates minority samples until balanced. Keeps all data but can cause over-fitting to duplicates.
SMOTE	Synthetic Minority Over-sampling TEchnique: creates new minority samples by interpolating between a minority sample and one of its k nearest minority neighbours.
ADASYN	Adaptive Synthetic sampling: like SMOTE but generates more synthetic samples for minority points that are harder to learn (many majority neighbours).
Decision Tree	Model that repeatedly splits the data on feature thresholds (Gini / entropy) to form a tree of if-then rules. Easy to interpret but prone to over-fitting.
Over-fitting	A model memorises noise in the training data: very high training score, noticeably lower test score.
Random Forest	Ensemble of many de-correlated decision trees (bagging + random feature subsets); majority vote reduces variance and improves generalisation.
Hyper-parameter	A setting chosen before training (e.g. max_depth, n_estimators) as opposed to parameters learned from data.
RandomizedSearchCV	Tries a fixed number of randomly sampled hyper-parameter combinations with cross-validation - efficient exploration of a large space.
GridSearchCV	Exhaustively tries every combination of a small grid with cross-validation - used for fine-tuning around a good region.
Cross-validation	Repeatedly training on k-1 folds and validating on the remaining fold to estimate performance without touching the test set.
Decision threshold	Probability cut-off above which a customer is flagged. Lowering it increases recall and false alarms; raising it does the opposite.